pi-iterative-goal

Operator and maintainer guide

PI EXTENSION GUIDE

Durable goal loops with governed tools, auditable state, and release authorization.

This guide covers the refactored pi-iterative-goal harness for operators and developers: how to start and control a goal, which tools are available, what the policy engine allows, how releases are authorized, and how to test the system without mutating real cloud or GitHub resources.

4
Required phases before evaluation

13
Approved OpenRouter model entries

18
Governed policy effects

0
External assets in this guide

Quick Start

Build the extension, let Pi load `dist/index.js`, then start a goal with an explicit success criterion. The loop runs `research -> plan -> implement -> validate`, then an external evaluator decides whether the goal is met.

```
npm install
npm run build

# In Pi, from the repository that should own the goal:
/goal-start Improve the checkout flow #criterion: npm run validate
passes and the checkout smoke path is documented
/goal-status --json
```

Normal Control

- Pause with `/goal-pause`; this is the normal stop control.
- Resume with `/goal-resume`; the harness rehydrates capabilities and sends a follow-up phase prompt.
- Reset with `/goal-reset`; the active run is archived before state is cleared.

Local Configuration

Project settings live in `.pi/settings.json`. AWS and git finalization default to disabled unless explicitly configured.

```
{
  "iterativeGoal": {
    "awsCli": { "enabled": false },
    "finalization": {
      "allowGitFinalization": false,
      "allowCommit": false,
      "allowPush": false,
      "allowPR": false,
      "fallback": "patch"
    }
  }
}
```

Workflow Model

The harness owns continuation. Agent output is captured on `agent_end`, phase artifacts are persisted, and only the evaluator can mark the goal as succeeded. A stale-write guard rejects tool reports that do not match the active `runId` and `phaseAttemptId` in the current `[HARNESS_META]` block.

Research

Collect context, capability inventory, blockers, and repo constraints.

Plan

Produce allowed paths, checks, risk notes, and any accepted plan amendments.

Implement

Change only planned paths. Implementation verification compares changed files to the approved plan scope.

Validate

Run tests and gates, persist verification results, then queue the external evaluator.

Evaluate

Evaluator returns strict JSON. `goal_met: true` is the only success signal.

`status: running` `status: paused_by_user` `status: recovering` `status: succeeded`

`directive: external_blocked_complete`

Command Reference

COMMAND	USE	NOTES
`/goal-start <goal> [#criterion: ...]`	Start or replace an autonomous run.	Preflights models and capabilities, then sends the research phase prompt.

COMMAND	USE	NOTES
`/goal-status [--json]`	Inspect active run, lock, evaluator state, artifacts, AWS and git policy.	`--json` is authoritative for automation.
`/goal-dashboard`	Show the interactive dashboard.	Requires interactive Pi UI support.
`/goal-pause`	Pause a running loop.	Use this instead of relying on model self-termination.
`/goal-resume`	Resume a paused loop.	Refreshes capability snapshot and starts a new phase attempt.
`/goal-repair-capabilities`	Re-run capability preflight and try model fallback repair.	Reports missing bash/subagent/AWS profile issues.
`/goal-finalize [--mode patch|commit]`	Write final patch artifacts or route to `goal_git` when commit mode is allowed.	Default behavior is patch-only.
`/goal-authorize-release`	Run the local pre-PR release authorization gate.	Requires evaluator success, no unresolved harness errors, clean local release gate evidence.
`/goal-audit`	Print run ID, event path, replay status, artifact path, and release authorization ID.	Fast operator handoff command.
`/goal-replay`	Replay the active event log and compare core state.	Legacy runs may be unavailable if they lack `run_created.initialState`.

COMMAND	USE	NOTES
`/goal-trace <term>`	Search run artifacts for a requirement or evidence term.	Returns matching phase/cycle/status lines.
`/goal-reset`	Archive and clear active run state.	Prompts before clearing.

Tool Reference

TOOL	INPUTS	BEHAVIOR
`goal_report_phase_result`	`runId`, `phaseAttemptId`, `phase`, `status`, `summary`	Mandatory phase completion report. Rejects stale run or phase IDs.
`goal_record_blocker`	`runId`, `phaseAttemptId`, `phase`, `title`, `description`, `severity`	Records a recoverable harness blocker for evaluator review.
`goal_request_capability_repair`	`what`, `kind`, optional `runId`	Records missing tools/MCP/model issues and can switch to a healthy fallback model.
`goal_checkpoint`	None	Forces state persistence.
`goal_shell`	`command`, optional `cwd`, `purpose`, `allowDestructive`	Runs parsed executable-plus-argv commands through safety and policy checks. Blocks package installs and routed AWS/git finalization.

TOOL	INPUTS	BEHAVIOR
`goal_aws_cli`	`args`, `purpose`, optional `profile`, `region`, `cwd`, `allowMutation`	Runs AWS CLI only when enabled, profile preflight passes, and mutation families are explicitly allowed.
`goal_git`	`action`, `purpose`, plus action-specific fields	Guards status, branch, add, commit, push, and PR creation. PR creation requires a current `ReleaseAuthorization`.
`goal_subagent`	`role`, `task`, optional `allowedPaths`, `model`, `mode`, `cwd`	Routes scouts/reviewers through detected backends. Writer roles require explicit allowed paths and isolated worktree leasing.

Connectivity Matrix

Capabilities are declared by provider manifests and enforced by `PolicyEngine` decisions. Browser, MCP, and vision providers are contract stubs unless a backend is configured.

SURFACE	CURRENT CONTRACT	POLICY BOUNDARY	SANDBOX STATUS
Filesystem	`filesystem.read`, `filesystem.write`, `filesystem.delete`	Writes/deletes must stay inside approved path scopes.	mocked pass
Process	`process.exec` and `goal_shell`	Executable-plus-argv only; dangerous commands and package installs are denied.	temp repo

SURFACE	CURRENT CONTRACT	POLICY BOUNDARY	SANDBOX STATUS
AWS CLI	`goal_aws_cli` plus local `.pi/settings.json`	Disabled by default; mutating families require config and `allowMutation=true`.	PATH shim
Git/GitHub CLI	`goal_git` for finalization actions	Commit/push/PR require finalization policy; PR also requires release authorization.	dry-run only
Subagents	`goal_subagent` with tool/Agent/command/none detection	Writer roles require `allowedPaths`; subprocesses are policy-brokered.	fallback checked
Web Fetch	`web.fetch` provider	Host allowlist, no redirects, public DNS check, private address denial.	no live fetch
Browser	`browser.interact` provider	Requires explicit browser approval and host allowlist; no backend by default.	contract only
MCP	`mcp.invoke` provider	Requires server/tool resource match and provider-scoped approval; no invoker by default.	contract only
Vision	`vision.inspect` provider	No network access; backend must be configured.	contract only

SURFACE	CURRENT CONTRACT	POLICY BOUNDARY	SANDBOX STATUS
Package Installs	`package.install` policy effect	Denied until represented in an approved plan with lockfile effects.	denied
CI/Cloud Placeholders	`ci.read`, `ci.trigger`, `cloud.read`, `cloud.mutate`, `secret.read` effects	Cloud mutations and secret reads require explicit operator approval.	policy only

Policy Model

Every governed action is represented as an `ActionRequest`. `PolicyEngine.decide()` returns a `PolicyDecision` of `allow`, `deny`, `require_approval`, or `transform`. Allowed actions receive a one-use `CapabilityLease` with a five-minute expiry.

Resource/Input Matching

Filesystem paths, process command specs, URLs, and MCP server/tool IDs must match both the resource descriptor and structured input.

Fail-Closed Network

Private, metadata-like, credential-bearing, unallowlisted, or redirecting URLs are denied for governed fetch/browser actions.

Finalization Separation

Git finalization is routed through `goal_git`; shell commands can read git state but cannot stage, commit, push, or create PRs when finalization is enabled.

```
ActionRequest {
  actor, runId, effect, resource, input,
  purpose, risk, dataClassification, allowedPaths?
}
```

```
PolicyDecision {
  result, ruleIds, reason, lease?, transformedInput?
}
```

Release Flow

PR creation is intentionally two-step. The run first earns a SHA-bound `ReleaseAuthorization`; only then can `goal_git` authorize `create_pr`. Any drift in HEAD, base, plan hash, requirement hash, gate verdict hash, or evidence root hash invalidates the authorization.

1. Evaluate

Current evaluator verdict must have `goal_met: true`.

2. Gate

Local release gate requires no unresolved harness errors, clean tree, implementation verification, and passing validation evidence.

3. Authorize

`/goal-authorize-release` records the current repository ID, base SHA, HEAD SHA, hashes, allowed action, and expiry.

4. Dry Run

`goal_git` can render the PR request with `dryRun: true` without calling GitHub.

5. Open PR

Actual PR creation additionally requires `gh` availability and authentication.

Agent Runtime

Read-Only Roles

Scout, requirements, planner, test, security, architecture, documentation, and release reviewer roles are treated as read-oriented by default.

Writer Roles

Implementer and Integrator roles require explicit `allowedPaths`; the subprocess task asks for an isolated worktree and limits permitted effects.

Concurrency rule: overlap is allowed for scouting and review, but writes must be scoped, leased, and validated against the plan before release.

Persistence And Audit

Runtime state is stored under `.pi/iterative-goal/`. New runs use run-scoped directories, append-only `events.jsonl`, and a hash chain with `sequence`, `previousEventHash`, and `eventHash`.

```
.pi/iterative-goal/
  active-run.json
  runs/<runId>/
    state.json
    events.jsonl
    latest.md
    evaluator-state.json
    evaluator-verdicts.jsonl
    cycles/<n>/research|plan|implement|validate/
```

Use `/goal-audit` for paths, `/goal-replay` for state reconstruction, and `/goal-trace ` to connect requirements to phase evidence.

Model Selection

The harness filters configured models against an approved OpenRouter allowlist. The default primary is `openrouter/deepseek/deepseek-v4-flash`; default fallbacks are

`openrouter/deepseek/deepseek-v4-pro`, `openrouter/anthropic/claude-sonnet-4.6`, and `openrouter/openrouter/fusion`.

ROLE	APPROVED MODELS
Primary	`deepseek/deepseek-v4-flash`
Fallback	`xiaomi/mimo-v2.5`, `minimax/minimax-m3`, `tencent/hy3-preview`, `openrouter/owl-alpha`, `z-ai/glm-5.2`
Evaluator	`deepseek/deepseek-v4-pro`
Reviewer	`anthropic/claude-opus-4.7`, `anthropic/claude-opus-4.8`, `anthropic/claude-sonnet-4.6`
Router	`openrouter/fusion`, `openrouter/pareto-code`, `openrouter/auto`

Sandbox Testing

The validation bundle uses disposable temp repos, mocked Pi extension APIs, mocked AWS CLI behavior, and policy-only negative cases. It does not create real GitHub PRs, mutate AWS, or write to cloud services.

```
node scripts/user-guide-sandbox-validation.mjs
```

Evidence Files

- sandbox-report.md
- sandbox-report.json
- screenshots/desktop.png
- screenshots/mobile.png

Expected Checks

- `npm run validate` passes.
- Extension registers commands and tools from `dist/index.js`.
- `goal_shell` runs safely in a temp git repo.

- Mock AWS and git/PR dry-run paths remain local.
- Package install, private URL, missing PR authorization, and stale phase writes are denied.

Troubleshooting

SYMPTOM	LIKELY CAUSE	OPERATOR ACTION
Missing Pi tools	Capability snapshot does not include expected tool names.	Run `/goal-repair-capabilities`; use `goal_shell` when bash is unavailable.
AWS profile issues	No CLI, no session-manager-plugin, or no usable profile.	Check `.pi/settings.json`, `AWS_PROFILE`, `aws sso login`, and `/goal-status --json` AWS fields.
Git finalization blocked	Policy is default-deny or git command was attempted through shell.	Enable only the needed finalization booleans and use `goal_git`; patch fallback remains available.
PR denied	No current `ReleaseAuthorization`, failed local gate, or SHA/hash drift.	Validate, run `/goal-authorize-release`, then retry `goal_git` with the matching authorization ID.
MCP/browser/vision unavailable	Provider contracts exist but no backend/invoker is configured.	Treat as fail-closed and document fallback evidence; do not imply live backend support.

SYMPTOM	LIKELY CAUSE	OPERATOR ACTION
Policy denial	Resource/input mismatch, private URL, package install, or missing approval flag.	Read `ruleIds` and `reason`; adjust the plan or request explicit operator approval where supported.

Residual Risks

Sandbox evidence proves local contracts, not real production infrastructure behavior. DNS rebinding, time-of-check/time-of-use drift, stale external credentials, and unavailable provider backends remain operational risks.

- Web fetch performs a public DNS check before fetch, but network destinations can still change after resolution.
- `ReleaseAuthorization` is SHA-bound, but it does not replace human review of the resulting PR body and diff.
- Browser, MCP, and vision providers intentionally fail closed without configured backends.
- Package installs remain denied until the harness has an approved plan and lockfile-aware capability path.

Generated for the refactored `pi-iterative-goal` harness. Source references are local TypeScript registrations, provider manifests, and the sandbox report in this directory.